

Learning-enabled Acceleration of Scenario-based Model Predictive Control

Trinh Tran, Binh Nguyen, Truong X. Nghiem

Abstract—Scenario-based model predictive control (SBMPC) is a variant of model predictive control (MPC) that explicitly accounts for uncertainty by optimizing control actions over multiple predicted scenarios. However, its computational complexity increases rapidly with the number of scenarios and prediction horizon, limiting its applicability to real-time planning and control. This paper presents a learning-accelerated Alternating Direction Method of Multipliers (ADMM) algorithm for efficiently solving SBMPC problems by leveraging parallel computing and Moreau envelope learning, while maintaining high solution accuracy. We reformulate the SBMPC problems into consensus forms that can be decomposed via ADMM, separating the scenario-dependent dynamics from non-anticipativity constraints and enabling parallel updates across scenarios and time steps. Building on this decomposition, we utilize a learning-to-optimize scheme that leverages Moreau envelope learning of the cost function to accelerate the primal update in ADMM, thereby reducing computation time. The proposed framework is evaluated on a microgrid energy management problem subject to load and renewable generation uncertainties. Comparisons with IPOPT and MadNLP, two popular and modern nonlinear programming solvers, demonstrate substantial computational speedups while maintaining reliable closed-loop control performance.

I. INTRODUCTION

Model Predictive Control (MPC) is a popular framework for constrained optimal control, which computes feedback actions by repeatedly solving a finite-horizon optimal control problem [1], [2]. In many applications, however, the future evolution of the system is affected by uncertain disturbances. These disturbances enter the predicted dynamics, constraints, and costs, and therefore must be accounted for when the controller plans actions over the horizon. This issue is especially important in applications such as microgrid energy management, where load demand and renewable generation fluctuate over time and directly affect the economic and operational performance of the closed-loop system [3].

Robust and stochastic MPC provide two common ways to handle uncertainty in finite-horizon control. Robust MPC provides strong constraint-satisfaction guarantees by optimizing against worst-case uncertainty realizations, but often yields conservative and economically inefficient control actions [4]. Stochastic MPC instead models uncertainty probabilistically and typically enforces constraints in expectation or through chance constraints [5]. This probabilistic

treatment can reduce conservatism, but the resulting chance-constrained problems can be nonconvex and computationally difficult to solve in real time [6].

Scenario-based MPC (SBMPC) offers a tractable deterministic approximation of stochastic MPC using sampled disturbance trajectories [7]. The scenario approach accommodates general disturbance distributions via sampling, but the problem grows quickly with both the prediction horizon and the number of scenarios. To handle this growing complexity, the resulting scenario programs are often solved using the Alternating Direction Method of Multipliers (ADMM) [8]. This method is particularly attractive because it can exploit the separable scenario structure by duplicating variables and enforcing non-anticipativity through consensus constraints. However, solving these primal updates across all scenarios at each iteration in real time remains a major computational bottleneck.

Learning-to-optimize has emerged as a promising tool for reducing the cost of repeated optimization in real-time decision-making [9]. For example, Baker [10] learned warm-start points for AC optimal power flow to improve convergence speed. Misra *et al.* [11] learned a small collection of active constraint sets for large-scale parametric optimization, thereby reducing online computation while preserving constraint satisfaction. Our previous work developed the Learning-Enabled ADMM Framework (LEAF) [12], which accelerates ADMM by learning the Moreau envelope associated with the primal update. By leveraging the Moreau envelope to smoothly approximate objectives and recover proximal updates, LEAF efficiently accelerates the repeated local minimizations in ADMM-based optimization.

In this paper, we develop a learning-accelerated ADMM method for solving SBMPC problems. The key idea is to first reformulate the SBMPC problem in a consensus form that decomposes the ADMM primal step into parallel subproblems across the scenario-time grid. We then apply LEAF to approximate these local primal updates, replacing costly repeated computations with efficient neural network evaluations. Our main contributions are as follows.

- (i) We formulate a consensus ADMM decomposition for SBMPC that decouples scenario-dependent dynamics and non-anticipativity constraints. The resulting primal updates are separable across scenarios and prediction time steps, which enables parallel computation over the full scenario-time grid.
- (ii) We integrate the Moreau-envelope learning framework LEAF into the decomposed ADMM algorithm to accelerate the local primal update subproblems. The learned

*This material is based upon work supported by the U.S. National Science Foundation under Grants No. 2449927 and No. 2514584.

All authors are with Department of Electrical and Computer Engineering, University of Central Florida, Orlando, FL 32816, USA. {tr586225, thanhbinh.nguyen, truong.nghiem}@ucf.edu

update can be reused across scenarios with the same local cost structure, avoiding scenario-specific retraining while reducing the dominant per-iteration computation.

- (iii) We evaluate the proposed learning-accelerated SBMPC method on a microgrid energy management problem with uncertain load demand and renewable generation. Numerical results show substantial speedups over IPOPT and MadNLP [13] while maintaining high solution quality and meaningful closed-loop control performance.

II. PROBLEM FORMULATION

We consider a discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t + d_t, \quad (1)$$

where $x_t \in \mathbb{R}^n$ denotes the system state vector, $u_t \in \mathbb{R}^m$ is the control input vector, $d_t \in \mathbb{R}^n$ represents an unmeasured disturbance vector with a possibly unknown distribution $\mathbb{D}_d$, and $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$ are the system matrices.

To account for disturbance in a tractable way, the possible future disturbance is approximated by a finite set of scenarios $s = 1, \dots, S$. Each scenario s captures a realization of the uncertain model parameters and disturbance sequence over the prediction horizon. We use $x_{s,k}$ and $u_{s,k}$ to denote the state and input predicted k steps ahead under scenario s at time t . Consider the following scenario MPC problem, where the disturbance is sampled from its distribution $\mathbb{D}_d$ to generate S scenarios of the disturbance sequence $\{d_{s,t+k}\}_{k=0}^{N-1}$:

$$\text{minimize } \frac{1}{S} \sum_{s=1}^S \left(\sum_{k=0}^{N-1} c_k(x_{s,k}, u_{s,k}) + c_N(x_{s,N}) \right) \quad (2a)$$

$$\text{s.t. } x_{s,k+1} = Ax_{s,k} + Bu_{s,k} + d_{s,t+k}, \quad (2b)$$

$$x_{s,0} = x_t, \quad \forall s = 1, \dots, S, \quad (2c)$$

$$(x_{s,k}, u_{s,k}) \in \mathcal{X} \times \mathcal{U}, \quad (2d)$$

$$\forall k = 0, \dots, N-1, s = 1, \dots, S,$$

$$P\mathbf{u} = 0, \quad (2e)$$

where x_t is the current state, N is the prediction horizon, $\mathbf{x}_s = (x_{s,1}, \dots, x_{s,N}) \in \mathbb{R}^{Nn}$, $\mathbf{u}_s = (u_{s,0}, \dots, u_{s,N-1}) \in \mathbb{R}^{Nm}$, $c_k(\cdot, \cdot)$ and $c_N(\cdot)$ are convex, possibly nonsmooth, stage and terminal cost functions, and $\mathcal{X}$ and $\mathcal{U}$ are the state and input constraint sets. If the matrix P is defined as

$$P = \begin{bmatrix} \mathbf{I}_{Nm} & -\mathbf{I}_{Nm} & 0 & 0 & \dots & 0 \\ 0 & \mathbf{I}_{Nm} & -\mathbf{I}_{Nm} & 0 & \dots & 0 \\ \dots & & & & & \\ 0 & 0 & \dots & 0 & \mathbf{I}_{Nm} & -\mathbf{I}_{Nm} \end{bmatrix},$$

then the non-anticipativity constraint (2e) enforces that $u_{1,k} = u_{2,k} = \dots = u_{S,k}$ for $k = 0, \dots, N-1$, i.e., the control input is scenario-independent. On the other hand, if

$$P = \begin{bmatrix} P_0 & -P_0 & 0 & 0 & \dots & 0 \\ 0 & P_0 & -P_0 & 0 & \dots & 0 \\ \dots & & & & & \\ 0 & 0 & \dots & 0 & P_0 & -P_0 \end{bmatrix},$$

where $P_0 = [\mathbf{I}_{n_u} \ 0 \ \dots \ 0]$, then the constraint (2e) enforces that $u_{1,0} = u_{2,0} = \dots = u_{S,0}$, i.e., the control input at the first step is scenario-independent.

The following standard assumptions are made.

Assumption 1: (i) The full system state is available at each time step t . (ii) The control input constraint set $\mathcal{U}$ is bounded and convex. (iii) The state constraint set $\mathcal{X}$ is convex. (iv) Convexity holds for the stage c_k and terminal c_N costs.

The dynamics constraint for each scenario s can be rewritten as $\mathbf{x}_s = M_A \mathbf{x}_s + M_B \mathbf{u}_s + M_0 x_t + \mathbf{d}_s$, or equivalently

$$M_G \begin{bmatrix} \mathbf{x}_s \\ \mathbf{u}_s \end{bmatrix} = -M_0 x_t - \mathbf{d}_s, \quad (3)$$

where $\mathbf{d}_s = (d_{s,t}, \dots, d_{s,t+N-1}) \in \mathbb{R}^{Nn}$, $M_G = \begin{bmatrix} M_A - \mathbf{I}_{Nn} & M_B \end{bmatrix} \in \mathbb{R}^{Nn \times (Nn + Nm)}$, with $M_A \in \mathbb{R}^{Nn \times Nn}$, $M_B \in \mathbb{R}^{Nn \times Nm}$, and $M_0 \in \mathbb{R}^{Nn \times n}$ defined as

$$M_A = \begin{bmatrix} 0 & & & & \\ A & 0 & & & \\ & \ddots & \ddots & & \\ & & A & 0 & \end{bmatrix}, \quad M_B = \mathbf{I}_N \otimes B, \quad M_0 = \begin{bmatrix} A \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Therefore, under Assumption 1, the dynamics constraint set

$$\mathcal{C}_s := \left\{ (\mathbf{x}_s, \mathbf{u}_s) \in \mathcal{X}^N \times \mathcal{U}^N \mid M_G \begin{bmatrix} \mathbf{x}_s \\ \mathbf{u}_s \end{bmatrix} = -M_0 x_t - \mathbf{d}_s \right\}$$

is convex and parameterized by x_t and $\mathbf{d}_s$.

Although the scenario-based formulation (2) provides a deterministic approximation for the stochastic MPC problem, solving it incurs significant computational overhead. This is because the size of the problem grows quickly with the number of scenarios, prediction horizon, and system dimensions. Furthermore, the coupling constraints (2e) and (3) require solving a full trajectory optimization for each scenario. This motivates a decomposition approach that splits across both scenarios and time steps to enable parallel computation.

III. LEARNING-ACCELERATED ADMM FOR SBMPC

A. ADMM algorithm for scenario-based MPC

We assume that all scenarios share the same stage and terminal cost functions, while the disturbance realizations differ across scenarios. We remove $\frac{1}{S}$ from the objective function for simplicity and rewrite (2) in consensus form as

$$\text{minimize } \sum_{s=1}^S \left(\sum_{k=0}^{N-1} c_k(x_{s,k}, u_{s,k}) + c_N(x_{s,N}) + I_{\mathcal{C}_s}(\mathbf{y}_s, \mathbf{v}_s) \right) + I_{\mathcal{P}}(\mathbf{p}_1, \dots, \mathbf{p}_S) \quad (4a)$$

$$\text{s.t. } \mathbf{x}_s = \mathbf{y}_s, \quad s = 1, \dots, S, \quad (4b)$$

$$\mathbf{u}_s = \mathbf{p}_s, \quad s = 1, \dots, S, \quad (4c)$$

$$\mathbf{v}_s = \mathbf{p}_s, \quad s = 1, \dots, S, \quad (4d)$$

where $\mathbf{y}_s \in \mathbb{R}^{Nn}$, $\mathbf{v}_s \in \mathbb{R}^{Nm}$, and $\mathbf{p}_s \in \mathbb{R}^{Nm}$ are auxiliary variables, and $\mathcal{P} = \{\mathbf{p} : P\mathbf{p} = 0\}$, $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_S)$, $\mathbf{p}_s \in \mathbb{R}^{Nm}$. Here, $I_{\mathcal{C}_s}(\mathbf{y}_s, \mathbf{v}_s)$ is the indicator function for the dynamics constraint set $\mathcal{C}_s$, which equals 0 if $(\mathbf{y}_s, \mathbf{v}_s) \in \mathcal{C}_s$, and $+\infty$ otherwise. Similarly, $I_{\mathcal{P}}(\cdot)$ is the indicator function for the set $\mathcal{P}$.

For the Lagrange multipliers $\lambda_{y,s}$ associated with $\mathbf{x}_s - \mathbf{y}_s = 0$, $\lambda_{v,s}$ associated with $\mathbf{v}_s - \mathbf{p}_s = 0$, and $\lambda_{p,s}$ associated with $\mathbf{u}_s - \mathbf{p}_s = 0$, and for a penalty parameter $\rho > 0$, the augmented Lagrangian is

$$\begin{aligned} \mathcal{L}_\rho(\mathbf{x}, \mathbf{u}, \mathbf{y}, \mathbf{v}, \mathbf{p}, \lambda_y, \lambda_v, \lambda_p) = & \sum_{s=1}^S \sum_{k=0}^{N-1} c_k(x_{s,k}, u_{s,k}) \\ & + \sum_{s=1}^S c_N(x_{s,N}) + \sum_{s=1}^S \mathcal{I}_{\mathcal{C}_s}(\mathbf{y}_s, \mathbf{v}_s) + \mathcal{I}_{\mathcal{P}}(\mathbf{p}_1, \dots, \mathbf{p}_S) \\ & + \sum_{s=1}^S (\lambda_{y,s}^\top (\mathbf{x}_s - \mathbf{y}_s) + \lambda_{v,s}^\top (\mathbf{v}_s - \mathbf{p}_s) + \lambda_{p,s}^\top (\mathbf{u}_s - \mathbf{p}_s)) \\ & + \frac{\rho}{2} \sum_{s=1}^S (\|\mathbf{x}_s - \mathbf{y}_s\|_2^2 + \|\mathbf{v}_s - \mathbf{p}_s\|_2^2 + \|\mathbf{u}_s - \mathbf{p}_s\|_2^2). \end{aligned}$$

The ADMM iterates for the scenario MPC problem (4) are

$$\begin{aligned} (x_{s,k}^{i+1}, u_{s,k}^{i+1}) = \arg \min_{x_{s,k}, u_{s,k}} & \left\{ c_k(x_{s,k}, u_{s,k}) \right. \\ & + \frac{\rho}{2} \|x_{s,k} - y_{s,k}^i + \rho^{-1} \lambda_{y,s,k}^i\|_2^2 \\ & + \frac{\rho}{2} \|u_{s,k} - p_{s,k}^i + \rho^{-1} \lambda_{p,s,k}^i\|_2^2 \Big\}, \\ & s = 1, \dots, S; \quad k = 0, \dots, N-1, \quad (5a) \end{aligned}$$

$$\begin{aligned} x_{s,N}^{i+1} = \arg \min_{x_{s,N}} & \left\{ c_N(x_{s,N}) \right. \\ & + \frac{\rho}{2} \|x_{s,N} - y_{s,N}^i + \rho^{-1} \lambda_{y,s,N}^i\|_2^2 \Big\}, \quad (5b) \end{aligned}$$

$$(\mathbf{y}_s^{i+1}, \mathbf{v}_s^{i+1}) = \Pi_{\mathcal{C}_s} \left(\begin{bmatrix} \mathbf{x}_s^{i+1} + \lambda_{y,s}^i \\ \mathbf{p}_s^i - \lambda_{v,s}^i \end{bmatrix} \right), \quad s = 1, \dots, S, \quad (5c)$$

$$\mathbf{p}^{i+1} = \Pi_{\mathcal{P}} \left(\frac{1}{2} (\mathbf{u}^{i+1} + \mathbf{v}^{i+1}) + \frac{1}{2\rho} (\lambda_p^i + \lambda_v^i) \right), \quad (5d)$$

$$\lambda^{i+1} = \lambda^i + \rho \left(\begin{bmatrix} \mathbf{x}^{i+1} \\ \mathbf{v}^{i+1} \\ \mathbf{u}^{i+1} \end{bmatrix} - \begin{bmatrix} \mathbf{y}^{i+1} \\ \mathbf{p}^{i+1} \\ \mathbf{p}^{i+1} \end{bmatrix} \right), \quad (5e)$$

where $\lambda = [\lambda_y^\top, \lambda_v^\top, \lambda_p^\top]^\top$ is the lumped Lagrange multiplier vector, $\Pi_{\mathcal{C}_s}(\cdot)$ and $\Pi_{\mathcal{P}}(\cdot)$ are the Euclidean projection operators onto sets $\mathcal{C}_s$ and $\mathcal{P}$, respectively.

These subproblems (5a) are separable in s and k , while the terminal updates (5b) are separable across scenarios. Therefore, they can be solved in parallel across scenarios and time steps. However, their computational cost can still be significant when the number of scenarios or the prediction horizon becomes large.

B. Moreau Envelope Learning for ADMM

In this part, we apply the Learning-Enabled ADMM Framework developed in our previous work [12] to accelerate the primal update step. Rather than directly solving the optimization problems in (5a) and (5b), LEAF leverages the Moreau envelope to provide a smooth, learnable approximation of the primal step.

The primal update can be equivalently written as the proximal operator of the local objective function as

$$z_{s,k}^{i+1} = \text{prox}_{\rho^{-1}c_k}(w_{s,k}^i), \quad k = 0, \dots, N, \quad s = 1, \dots, S$$

where $z_{s,k}^{i+1} = [x_{s,k}^{i+1}, u_{s,k}^{i+1}]^\top$, $w_{s,k}^i = [y_{s,k}^i, p_{s,k}^i]^\top - \rho^{-1}[\lambda_{y,s,k}^i, \lambda_{p,s,k}^i]^\top$. Note that $u_{s,N} = p_{s,N} = \lambda_{p,s,N} = 0_m$, $s = 1, \dots, S$. For more details on proximal operators and their role in convex optimization, we refer readers to [14].

The corresponding Moreau envelope [15] of c_k with parameter ρ^{-1} is defined as

$$M_{\rho^{-1}c_k}(w_{s,k}^i) = \min \left\{ c_k(z_{s,k}) + \frac{\rho}{2} \|z_{s,k} - w_{s,k}^i\|_2^2 \right\}.$$

The proximal operator and the Moreau envelope satisfy the following relationship

$$\text{prox}_{\rho^{-1}f}(w) = w - \rho^{-1} \nabla M_{\rho^{-1}f}(w). \quad (6)$$

Eventually, the primal update can be equivalently written as

$$z_{s,k}^{i+1} = w_{s,k}^i - \rho^{-1} \nabla M_{\rho^{-1}c_k}(w_{s,k}^i).$$

The Moreau envelope preserves the convexity of c_k while providing a differentiable approximation, even when c_k is nonsmooth. Therefore, instead of directly learning the solutions of the nonsmooth optimization problem (5a), (5b), our learning-based method approximates the gradient of the Moreau envelope and uses it to recover the primal update.

For any observed input $w_{s,k}^i$, we train an input-convex neural network model $\mathcal{N}_{\theta,k}$ with weights θ_k to approximate the Moreau envelope $M_{\rho^{-1}c_k}(w_{s,k}^i)$. Equivalently, the neural network outputs $\hat{M}_{\theta,k}(w_{s,k}^i) = \mathcal{N}_{\theta,k}(w_{s,k}^i)$, and from that we can obtain its gradient $\nabla \hat{M}_{\theta,k}(w_{s,k}^i)$ according to (6). The gradient of the learned envelope is then used to reconstruct a primal update as follows

$$\hat{z}_{s,k}^{i+1} = w_{s,k}^i - \rho^{-1} \nabla \hat{M}_{\theta,k}(w_{s,k}^i),$$

where $\hat{z}_{s,k}^{i+1} = [\hat{x}_{s,k}^{i+1}, \hat{u}_{s,k}^{i+1}]^\top$ with $\hat{x}_{s,k}^{i+1}$ and $\hat{u}_{s,k}^{i+1}$ are the approximated primal variables. Thus, the expensive solution of the local unconstrained optimization problem in (5a) is replaced by a forward evaluation of the neural network and a gradient computation. Once trained, the learned update can be reused across ADMM iterations and across scenario-time blocks that share the same local cost structure.

The above training goal could be realized by minimizing the following loss function for the neural network $\mathcal{N}_{\theta,k}$ in a supervised-learning manner:

$$\begin{aligned} L_k(\theta_k) = \sum_{j=1}^{n_D} & \left(\left\| \hat{M}_{\theta,k}(w_j) - M_{\rho^{-1}c_k}(w_j) \right\|_2^2 \right. \\ & + \alpha \left\| \nabla \hat{M}_{\theta,k}(w_j) - \nabla M_{\rho^{-1}c_k}(w_j) \right\|_2^2 \\ & \left. + \beta \left\| \max \left(0, \hat{M}_{\theta,k}(w_j) - c_k(w_j) \right) \right\|_2^2 \right), \quad (7) \end{aligned}$$

where $\alpha > 0$ and $\beta > 0$ are weighting coefficients, n_D is the number of supervised training samples, and j indexes each training sample. The last term penalizes violations of the Moreau-envelope property $M_{\lambda f}(x) \leq f(x)$ for all x , by enforcing $\hat{M}_{\theta,k}(w_j) \leq c_k(w_j)$ on the training samples. The scenario index s does not appear explicitly in (7) because the network is trained on local inputs w_j drawn from the collection of inputs $w_{s,k}^i$ across scenarios, time steps, and

ADMM iterations. After collection, each w_j is treated as one sample for the learning map associated with c_k .

IV. EXPERIMENTAL RESULTS

In this section, we evaluate the proposed method on the optimal control task of energy management in microgrids.

A. System description

We consider the optimal energy management of a microgrid with renewable energy integration and battery energy storage system (BESS) [16]. The system consists of controllable loads, uncontrollable loads, a BESS, a renewable generator, and a utility-grid connection. The objective is to schedule the BESS charging/discharging power to maximize the average profit.

The uncertainty in the grid arises from fluctuations in renewable energy generation and load demand. We represent this uncertainty by a finite scenario set $\mathcal{S} = \{1, \dots, S\}$. We formulate the operation problem for the microgrid over the planning horizon N with time step Δt . For each scenario $s \in \mathcal{S}$, the net uncontrollable power disturbance at step k

$$d_{s,k} = P_{s,k}^u - P_{s,k}^R, \quad (8)$$

where $P_{s,k}^u \geq 0$ is the uncontrollable load and $P_{s,k}^R \geq 0$ is the renewable generation. A positive $d_{s,k}$ means uncontrollable demand exceeds renewable supply. The state is the BESS state of charge (SOC) $q_{s,k}$, and the control $u_k = [P_k^c, P_k^B]^\top$ is the scenario-independent schedule. P_k^c is the controllable load and P_k^B is the BESS charging/discharging power. A positive P_k^B represents discharging and a negative value represents charging. The SOC dynamics for scenario s are

$$q_{s,k+1} = q_{s,k} - \frac{\Delta t}{b} P_k^B, \quad (9)$$

The schedule is scenario-independent because decisions must be made before the uncertainty is realized (non-anticipativity). Note that the disturbance $d_{s,k}$ does not appear in (9). Instead, it enters through the power balance $P_k^c + P_{s,k}^u = D_{s,k} + P_k^B + P_{s,k}^R$, which gives the net power drawn from the utility grid:

$$D_{s,k} = P_k^c + d_{s,k} - P_k^B. \quad (10)$$

The stage cost for scenario s at step k is the sum of an energy cost, a peak-demand cost, and a discomfort cost:

$$c_k(q_{s,k}, u_k, d_{s,k}) = p_k \Delta t \left(D_{s,k} + \frac{1-\mu}{2\sqrt{\mu}} |P_k^B| \right) + p_p \max(D_{s,k}, 0) + \eta g(P_k^c), \quad (11)$$

where p_k is the electricity price, $\mu \in (0, 1]$ is the round-trip efficiency of the BESS, $p_p \geq 0$ weights the peak-demand charge, and $\eta \geq 0$ weights the discomfort penalty. The term $\frac{1-\mu}{2\sqrt{\mu}} |P_k^B|$ accounts for the energy lost due to BESS cycling. The convex discomfort function $g(\cdot)$ penalizes consumption below the preferred level γ :

$$g(P) = \begin{cases} +\infty, & P \leq 0, \\ \frac{\gamma}{P} - 1, & 0 < P < \gamma, \\ 0, & P \geq \gamma. \end{cases}$$

The scenario MPC problem for energy management is

$$\text{minimize } \frac{1}{S} \sum_{s=1}^S \sum_{k=0}^{N-1} c_k(q_{s,k}, u_k, d_{s,k}) \quad (12a)$$

$$\text{s.t. } q_{s,k+1} = q_{s,k} - \frac{\Delta t}{b} P_k^B, \quad (12b)$$

$$q_{s,0} = q_t, \quad s = 1, \dots, S, \quad (12c)$$

$$B_{\min} \leq P_k^B \leq B_{\max}, \quad k = 0, \dots, N-1, \quad (12d)$$

$$Q_{\min} \leq q_{s,k} \leq Q_{\max}, \quad (12e)$$

$$Q_N \leq q_{s,N}, \quad s = 1, \dots, S, \quad (12f)$$

$$P\mathbf{u} = 0, \quad (12g)$$

where $B_{\max} > 0$ is the maximum discharge power, $B_{\min} < 0$ is the maximum charge power, $Q_{\min}$ and $Q_{\max}$ are the SOC limits, and Q_N is the minimum terminal SOC reserve. The non-anticipativity constraint (12g) enforces a scenario-independent schedule, following the structure of (2e).

In order to apply the proposed framework for the formulation (12), we need to reformulate it to fit the general scenario MPC. The stage cost (11) depends explicitly on the disturbance $d_{s,k}$ through $D_{s,k}$, whereas the general scenario MPC (2) requires the stage cost $c_k(\cdot, \cdot)$ to depend only on the state and control. This mismatch arises because the SOC dynamics (9) are disturbance-free, i.e., the disturbance does not enter the state trajectory and therefore cannot be captured by the state alone. We augment the state with the disturbance $d_{s,k}$, thereby moving it from the stage cost into the dynamics.

Define the augmented state $\tilde{x}_{s,k} = [q_{s,k}, d_{s,k}]^\top$, the augmented dynamics then take the linear form

$$\tilde{x}_{s,k+1} = \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}}_{\tilde{A}} \tilde{x}_{s,k} + \underbrace{\begin{bmatrix} 0 & -\frac{\Delta t}{b} \\ 0 & 0 \end{bmatrix}}_{\tilde{B}} u_k + \underbrace{\begin{bmatrix} 0 \\ d_{s,k+1} \end{bmatrix}}_{\tilde{d}_{s,k}}.$$

The exogenous term $\tilde{d}_{s,k} = [0, d_{s,k+1}]^\top$ is the disturbance that enters the augmented dynamics at step k , corresponding to $d_{s,t+k}$ in the general formulation (2b). With the augmented state, the stage cost (11) depends on $\tilde{x}_{s,k}$ and $u_{s,k}$ only:

$$c_k(\tilde{x}_{s,k}, u_{s,k}) = p_k \Delta t \left(P_k^c + d_{s,k} - P_k^B + \frac{1-\mu}{2\sqrt{\mu}} |P_k^B| \right) + p_p \max(P_k^c + d_{s,k} - P_k^B, 0) + \eta g(P_k^c), \quad (13)$$

where $d_{s,k} = [\tilde{x}_{s,k}]_2$ is read from the second component of the augmented state. This matches the form $c_k(x_{s,k}, u_{s,k})$ required by (2). The augmented initial condition is

$$\tilde{x}_{s,0} = [q_t, d_{s,0}]^\top, \quad (14)$$

where q_t is the current SOC (scenario-independent) and $d_{s,0}$ is the initial disturbance realization for scenario s . The scenario-dependent component $d_{s,0}$ can be viewed as arising from the exogenous injection $\tilde{d}_{s,-1} = [0, d_{s,0}]^\top$ applied to the common base state $\tilde{x}_t = [q_t, 0]^\top$ before the first horizon step. This is the scenario-dependent initial condition $x_{s,0}$.

In summary, the energy management scenario MPC (12) with the augmented state is an instance of the general scenario MPC (2) with $A = \tilde{A}$, $B = \tilde{B}$, $d_{s,t+k} = \tilde{d}_{s,k}$, $x_{s,0} = \tilde{x}_{s,0}$, $x_{s,k} = \tilde{x}_{s,k}$, cost function (13), input constraint (12d), state constraints (12e)–(12f), and non-anticipativity constraint (12g).

TABLE I: Training configuration and architecture parameters of the input-convex neural network.

Setting	Value	Setting	Value
Optimizer	AdamW	Train/test instances	8000 / 2000
Learning rate	10^{-3}	Hidden layers	16×16
Epochs	5000	Activation	Softplus

B. Experiment setting

The problem parameters are set based on [16] as: $\Delta t = 0.25$ h, $p_k = \$0.1/\text{kWh}$, $p_p = \$19.19/\text{kW}$, $\eta = \$10$, $q_{s,0} = 0.5$, $B_{\min} = -700$ kW, $B_{\max} = 700$ kW, $b = 500$ kWh, $\mu = 0.8$, $Q_{\min} = 0.2$, $Q_{\max} = 0.8$, and $\gamma = 50$ kW. We perform a 10-step closed-loop MPC simulation with horizon $N \in \{48, 96\}$ and scenario $S \in \{10, 100\}$. Table I summarizes the training and architecture hyperparameters used for the neural network.

To evaluate the performance of the proposed learning-accelerated SBMPC method, abbreviated as LA-SBMPC, and compare it against IPOPT and MadNLP [13], we implement all algorithms in Julia on an Apple M4 Pro chip with 24 GB of RAM.

C. Scenario construction and sampling

Recall that the uncertainties in (12) are the uncontrollable load P^u and photovoltaic (PV) renewable generation P^R . Following the scenario MPC formulation in (2), each scenario must provide one realization of these uncertain signals over the prediction horizon. We use the 24-hour real-world data from [16], sampled every $\Delta t = 0.25$ h, and repeat the profile over 7 days with additive Gaussian perturbations of standard deviation 1 kW to construct the forecasting data. This produces a scenario set with $L = 7 \times 96$ time steps.

For each signal type $\zeta \in \{u, R\}$, a Gaussian process (GP) model [17] is fitted using time as the input, a constant mean function, and a periodic kernel with a 24-hour period. The fitted GP provides the joint predictive distribution over the full scenario horizon,

$$\tilde{\mathbf{P}}_s^\zeta \sim \mathcal{N}(\boldsymbol{\mu}^\zeta, \boldsymbol{\Sigma}^\zeta), \quad s = 1, \dots, S,$$

where $\boldsymbol{\mu}^\zeta \in \mathbb{R}^L$ and $\boldsymbol{\Sigma}^\zeta \in \mathbb{R}^{L \times L}$ are the GP posterior mean and covariance evaluated at the 15-minute time grid. The final scenario values are projected onto the nonnegative range $P_{s,k}^\zeta = \max\{0, \tilde{P}_{s,k}^\zeta\}$, to enforce nonnegative load and generation values.

At each MPC sampling time t , we randomly select S trajectories from this scenario set and extract the prediction window from t to $t + N - 1$. Thus, the load and PV used by the MPC at a given start time t are

$$\mathbf{P}_t^u = P_{i,t:t+N-1}^u, \quad \mathbf{P}_t^R = P_{i,t:t+N-1}^R,$$

where i denotes the randomly selected scenario indices. Therefore, the length of each scenario window is determined by the MPC horizon N . In the simulations, for each sampled scenario and each MPC window, the disturbance sequence is obtained from these two signals using (8) and is passed to the optimization model through the power balance constraint (10).

TABLE II: Average per-step closed-loop solving time (milliseconds) comparison.

Scenario	IPOPT	MadNLP	LA-SBMPC
$S = 10, N = 48$	202	226	42
$S = 100, N = 48$	1879	6046	355
$S = 10, N = 96$	439	713	92
$S = 100, N = 96$	4632	11216	548

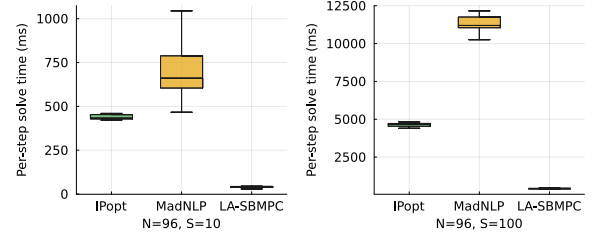


Fig. 1: Per-step closed-loop solving time comparison.

D. Results

Table II compares the average per-step closed-loop solving time under different numbers of scenarios and prediction horizons. In the smallest setting with $S = 10, N = 48$, the proposed LA-SBMPC requires only 42 ms, which is about $4.8\times$ faster than IPOPT and $5.4\times$ faster than MadNLP. When the prediction horizon is increased to $N = 96$ while keeping $S = 10$, LA-SBMPC maintains its advantage, requiring 92 ms and achieving speedups of $4.8\times$ and $7.8\times$ over IPOPT and MadNLP, respectively. As the number of scenarios scales up to $S = 100$, the computational benefits of LA-SBMPC become even more significant. Across the $S = 100$ settings, LA-SBMPC achieves speedups ranging from $5.3\times$ to $8.5\times$ against IPOPT, and from $17\times$ to $20.5\times$ against MadNLP, with a maximum average per-step solve time of just 548 ms. These results suggest that LA-SBMPC scales well as the number of scenarios and the prediction horizon increase. Moreover, Fig. 1 provides a per-step view of the closed-loop solving time, showing results consistent with Table II. Specifically, LA-SBMPC achieves a clear runtime advantage in the larger settings. As the number of scenarios and the prediction horizon increase, the baseline solvers (IPOPT and MadNLP) require substantially more time at each MPC step, whereas LA-SBMPC grows more moderately.

Table III and Fig. 2 evaluate the accuracy and feasibility of LA-SBMPC. The mean optimality gap is below 0.017% in all cases, and the maximum gap is below 0.081%, showing close agreement with IPOPT. The SOC, terminal, and non-anticipativity residuals are all small, confirming that LA-SBMPC maintains feasibility across the tested horizons and scenario counts. Fig. 3 illustrates the closed-loop performance of the microgrid over a 24-hour period under the LA-SBMPC schedule. The figure includes: (a) the grid power exchange, (b) the BESS charging/discharging power, (c) the controllable load, (d) the BESS state of charge, (e) the uncontrollable load, and (f) the renewable generation. The resulting trajectories are physically reasonable and capture the main operating characteristics of the microgrid. In particular, the BESS charges or discharges in response to

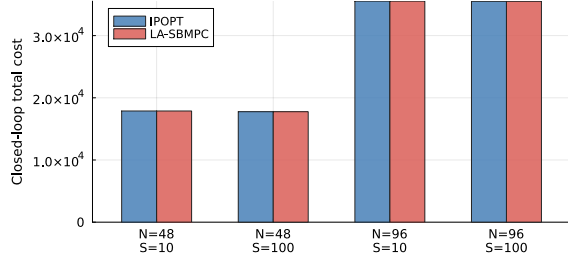


Fig. 2: Closed-loop total cost comparison under varying scenario and horizon settings.

TABLE III: Optimality gaps and constraint residuals of LA-SBMPC over all benchmark scenarios.

(N, S)	Optimality gap (%) mean (max)	SOC residual	Terminal residual	Non-anticip. residual
(48, 10)	2.40×10^{-3} (5.00×10^{-3})	1.48×10^{-13}	1.67×10^{-15}	2.17×10^{-15}
(48, 100)	6.00×10^{-3} (9.40×10^{-3})	4.83×10^{-15}	1.11×10^{-16}	1.32×10^{-14}
(96, 10)	1.56×10^{-2} (8.00×10^{-2})	8.28×10^{-13}	7.62×10^{-14}	7.11×10^{-15}
(96, 100)	1.61×10^{-2} (8.04×10^{-2})	1.30×10^{-12}	8.30×10^{-14}	1.42×10^{-14}

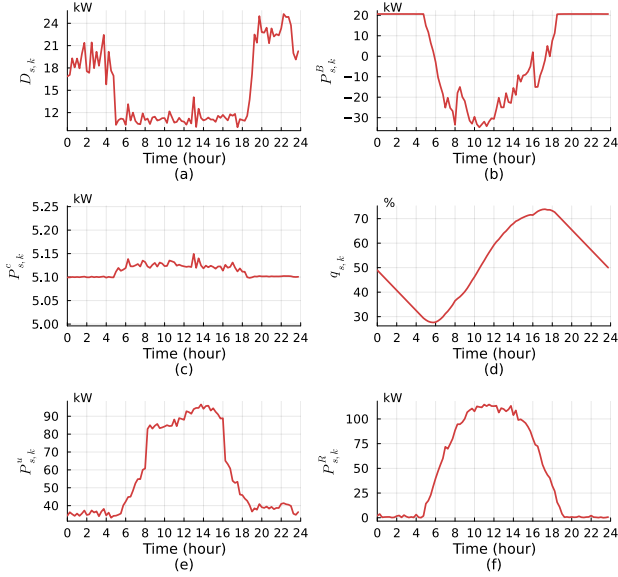


Fig. 3: Closed-loop 24-hour operation of the microgrid scheduled by LA-SBMPC for $S = 10$.

variations in load and renewable generation, while the SOC remains within its prescribed bounds. The controllable load is also adjusted according to system conditions, and the grid exchange reflects the balance between demand, renewable supply, and battery operation. These results demonstrate that the proposed method can produce meaningful closed-loop control actions for microgrid energy management over a period of time.

V. CONCLUSION

In this paper, we accelerated the solution of computationally demanding scenario-based MPC problems by developing a learning-based ADMM framework. By reformulating the scenario optimal control problem into a consensus form, we decoupled the coupling constraints and decomposed the problem via ADMM. To overcome the latency of repeated primal updates within this structure, we integrated

an input-convex neural network architecture to approximate the Moreau envelope of the local cost functions, thereby reducing the per-iteration computational overhead while strictly preserving the convexity of the problem. The efficacy of this approach was demonstrated on a microgrid energy scheduling problem subject to load and renewable generation uncertainties. Our empirical results show that the proposed algorithm achieves runtime speedups compared to popular nonlinear programming solvers. Future work will focus on learning warm-start solutions and considering closed-loop stability guarantees.

REFERENCES

- [1] C. E. Garcia, D. M. Prett, and M. Morari, “Model predictive control: Theory and practice—a survey,” *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.
- [2] N. T. Nguyen, B. Nguyen, A. Amine, T. Vo-Duy, R. Mangharam, and T. X. Nghiem, “AD-MPCC: Adaptive differentiable model predictive contouring control for autonomous racing,” *arXiv preprint arXiv:2607.00141*, 2026.
- [3] Z. Zhao, J. Xu, J. Guo, Q. Ni, B. Chen, and L. L. Lai, “Robust energy management for multi-microgrids based on distributed dynamic tube model predictive control,” *IEEE Transactions on Smart Grid*, vol. 15, no. 1, pp. 203–217, 2023.
- [4] A. Bemporad and M. Morari, “Robust model predictive control: A survey,” in *Robustness in identification and control*, A. Garulli and A. Tesi, Eds. London: Springer London, 1999, pp. 207–226.
- [5] A. Mesbah, “Stochastic model predictive control: An overview and perspectives for future research,” *IEEE Control Systems Magazine*, vol. 36, no. 6, pp. 30–44, 2016.
- [6] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming*. Society for Industrial and Applied Mathematics, 2009.
- [7] F. Micheli and J. Lygeros, “Scenario-based stochastic mpc for systems with uncertain dynamics,” in *2022 European Control Conference (ECC)*. IEEE, 2022, pp. 833–838.
- [8] J. Kang, A. U. Raghunathan, and S. Di Cairano, “Decomposition via admm for scenario-based model predictive control,” in *2015 American Control Conference (ACC)*. IEEE, 2015, pp. 1246–1251.
- [9] T. Chen, X. Chen, W. Chen, H. Heaton, J. Liu, Z. Wang, and W. Yin, “Learning to optimize: A primer and a benchmark,” *Journal of Machine Learning Research*, vol. 23, no. 189, pp. 1–59, 2022.
- [10] K. Baker, “Learning warm-start points for ac optimal power flow,” in *2019 IEEE 29th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, 2019, pp. 1–6.
- [11] S. Misra, L. Roald, and Y. Ng, “Learning for constrained optimization: Identifying optimal active constraint sets,” *INFORMS Journal on Computing*, vol. 34, no. 1, pp. 463–480, 2022.
- [12] B. Nguyen, T. Tran, and T. X. Nghiem, “LEAF: A learning-enabled ADMM framework for accelerated convex optimization,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.08993>
- [13] S. Shin, C. Coffrin, K. Sundar, and V. M. Zavala, “Graph-based modeling and decomposition of energy infrastructures,” *IFAC-PapersOnLine*, vol. 54, no. 3, pp. 693–698, 2021.
- [14] N. Parikh and S. Boyd, “Proximal algorithms,” *Foundations and Trends in Optimization*, vol. 1, no. 3, p. 127–239, 2014.
- [15] H. H. Bauschke and P. L. Combettes, *Convex analysis and monotone operator theory in Hilbert spaces*, 2nd ed. Springer Cham, 2020.
- [16] C. Cortes-Aguirre, Y.-A. Chen, A. Ghosh, J. Kleissl, and A. Khurram, “Economic MPC with an Online Reference Trajectory for Battery Scheduling Considering Demand Charge Management,” *IEEE Transactions on Smart Grid*, pp. 1–1, 2025.
- [17] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2, no. 3.
- [18] N. T. Nguyen, J. C. Tique, and T. X. Nghiem, “Physics-constrained taylor neural networks for learning and control of dynamical systems,” in *2025 American Control Conference (ACC)*. IEEE, 2025, pp. 1821–1826.